B.Tech. BCSE497J - Project-I

TRUSTSPHERE AI: A MULTI-AGENT FRAMEWORK FOR IMAGE MANIPULATION DETECTION WITH VERIFIABLE DIGITAL EVIDENCE

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

23BCI0065 Divyansh Gupta

23BCI0160 Shubham Raj

23BCI0053 Rudrarka Nath

Under the Supervision of

MANIKANDAN G

Assistant Professor Sr. Grade 2

School of Computer Science and Engineering (SCOPE)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

DECLARATION

We hereby declare that the project entitled TrustSphere AI: A Multi-Agent Framework for Image Manipulation Detection with Verifiable Digital Evidence submitted by us, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering* to VIT is a record of bonafide work carried out by us under the supervision of Prof. Manikandan G.

We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree ordiploma in this institute or any other institute or university.

Place : Vellore

Date :

Signature of the Candidates

CERTIFICATE

This is to certify that the project entitled TrustSphere AI: A Multi-Agent Framework for Image Manipulation Detection with Verifiable Digital Evidence submitted by Divyansh Gupta (23BCI0065), Shubham Raj (23BCI0160) and Rudrarka Nath (23BCI0053), **School of Computer Science and Engineering**, VIT, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering*, is a record of bonafide work carried out by them under my supervision during Fall Semester 2026-2027, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute orany other institute or university. The project fulfills the requirements and regulations ofthe University and in my opinion meets the necessary standards for submission.

Place : Vellore

Date :

Signature of the Guide

Examiner(s)

Dr. RAJESHKANNAN R, Head of the Department

B.Tech

ACKNOWLEDGEMENTS

We thank Vellore Institute of Technology (VIT) for giving us the chance and the resources to carry out this project. The facilities and support at VIT helped us build and test our ideas.

We sincerely thank Dr. Jaisankar N, Dean of the School of Computer Science and Engineering (SCOPE), for the support and encouragement given to the students of the school. It motivated us to do our best work.

We thank Dr. Rajeshkannan R, Head of the Department, for guidance and support during this project. The advice and feedback we received helped us shape the work and solve problems along the way.

We are very thankful to our project guide, Manikandan G, Assistant Professor Sr. Grade 2, for regular guidance and useful feedback. The time given in our reviews helped us improve our methods and finish this work.

Thank you all for your support.

Divyansh Gupta, Shubham Raj and Rudrarka Nath

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	**x**
1.	**INTRODUCTION**	**1**
	1.1 Background	**1**
	1.2 Motivations	**1**
	1.3 Scope of the Project	**1**
2.	**PROJECT DESCRIPTION AND GOALS**	2
	2.1 Literature Review	2
	2.2 Research Gap	4
	2.3 Objectives	4
	2.4 Problem Statement	4
	2.5 Project Plan	4
3.	**TECHNICAL SPECIFICATION**	6
	3.1 Requirements	6
	3.1.1 Functional	6
	3.1.2 Non-Functional	6
	3.2 Feasibility Study	6
	3.2.1 Technical Feasibility	6
	3.2.2 Economic Feasibility	6
	3.2.3 Social Feasibility	7
	3.3 System Specification	7
	3.3.1 Hardware Specification	7
	3.3.2 Software Specification	7
4.	**DESIGN APPROACH AND DETAILS**	8
	4.1 System Architecture	8
	4.2 Design	8
	4.2.1 Data Flow Diagram	8
	4.2.2 Use Case Diagram	9
	4.2.3 Class Diagram	9
	4.2.4 Sequence Diagram	10
5.	**METHODOLOGY AND TESTING**	12
	5.1 Module Description	12

	5.2 Testing	14
6.	**PROJECT DEMONSTRATION**	15
7.	**RESULT AND DISCUSSION (COST ANALYSIS as applicable)**	
8.	**CONCLUSION**	27
9.	**REFERENCES**	28
	APPENDIX A – SAMPLE CODE	32

List of Figures

Figure No.	Title	Page No.
4.1	System architecture of TrustSphere AI	8
4.2	Data-flow diagram, level 1	8
4.3	Use-case diagram	9
4.4	Class diagram: evidence data model	10
4.5	Class diagram: agents and decision engine	10
4.6	Sequence diagram: analysing one image	11
5.1	Detection pipeline of the trained image forensics agent	12
6.1	Analyze page: the user chooses an image and sees the analysis stages	15
6.2	A file selected for analysis	15
6.3	Verdict card for a tampered image	16
6.4	Agent breakdown for the tampered image: metadata, image forensics with heatmaps, provenance	17
6.5	Evidence chain: file integrity, Ed25519 signature, anchor status and chain of custody	18
6.6	Verdict cards of a second tampered image (Fake), an authentic image (Authentic) and an authentic image the engine sends to human review (Inconclusive)	18
6.7	History page: every stored analysis with its verdict and score	19
6.8	How it works page: the agents, the fusion weights and the evidence chain	20
6.9	First page of the evidence PDF of a tampered image	21
7.1	CASIA v2.0, random 5-fold cross-validation: confusion matrix, ROC curve and score distribution	22
7.2	CASIA v2.0, source-disjoint 5-fold cross-validation	22
7.3	Pipeline verdicts on the held-out images	23
7.4	Hold-out detector probability before the decision engine	23
7.5	Sample outputs from the held-out set: input, error-level analysis and Grad-CAM	24
7.6	Columbia, whole-image detector with ELA input	25
7.7	Columbia, patch-based detector with SRM input	25
7.8	Columbia, patch-based detector with scenes kept apart	25

List of Tables

Table No.	Title	Page No.
2.1	Summary of the base papers	3
2.2	Project plan	4
3.1	Hardware used for development, training and testing	7
3.2	Software stack	7
5.1	End-to-end test results	14
7.1	Detector results on CASIA v2.0	22
7.2	Decision-engine verdicts on the held-out images	23
7.3	Columbia, 5-fold cross-validation (all variants tried are listed)	24
7.4	Localiser trained on CASIA v2.0	25
7.5	CASIA v2.0 results of the base papers that the trained detector exceeds	26
7.6	Columbia results of earlier methods listed in [1] that the patch-based detector exceeds	26
7.7	Cost of building and running the system	26

List of Abbreviations

API	Application Programming Interface
AUC	Area Under the Curve
C2PA	Coalition for Content Provenance and Authenticity
CASIA	Institute of Automation, Chinese Academy of Sciences (image tampering database)
CNN	Convolutional Neural Network
CV	Cross-Validation
DFD	Data-Flow Diagram
ELA	Error Level Analysis
EER	Equal Error Rate
EXIF	Exchangeable Image File Format
FFT	Fast Fourier Transform
GPU	Graphics Processing Unit
JPEG	Joint Photographic Experts Group
JSON	JavaScript Object Notation
PDF	Portable Document Format
PNG	Portable Network Graphics
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SHA	Secure Hash Algorithm
SRM	Spatial Rich Model (noise-residual filters)
UML	Unified Modeling Language
UUID	Universally Unique Identifier

Symbols and Notations

p	Predicted probability that an image is manipulated
$s = 1 - p$	Authenticity score of the detector
S	Fused authenticity score of the decision engine
w_a	Weight of agent a in the fused score
$T_A = 0.75$	Authentic threshold
$T_F = 0.40$	Fake threshold

ABSTRACT

Photographs are used as evidence in news, insurance, banking and courts. Editing tools make it easy to paste part of one image into another. Deciding whether an image was changed is one problem. Proving later that the decision was not altered is another. This project builds TrustSphere AI, a system for still images that handles both. Three agents look at each uploaded image. A metadata agent reads the EXIF data. An image forensics agent studies the pixels. A provenance agent checks content credentials and invisible watermarks. A decision engine combines their scores into a verdict of Authentic, Fake or Inconclusive. Each result is hashed with SHA-256, signed with Ed25519, stored with a chain of custody, anchored on a test blockchain if a wallet is set, and saved as an evidence PDF. A web application shows the whole process.

The image forensics agent uses a detector we trained. It is an EfficientNet-B0 network that reads error level analysis maps made at three JPEG qualities, using crops at the original resolution. We trained and tested it on a subset of CASIA v2.0 (1,536 authentic and 1,283 tampered images) and on Columbia (183 authentic and 180 spliced images). With 5-fold cross-validation on CASIA v2.0 the accuracy is 95.53% with a random split and 95.32% when each tampered image stays in the same fold as its source. On 588 images that were never used in training it reaches 96.26% and the full pipeline gives a verdict on 90.5% of them with 97.9% accuracy. The rest go to a person. Columbia is harder because its photos are uncompressed. A whole image detector reaches only 87.60%, while a patch based detector on noise residuals reaches 97.25% with a random split and 96.97% when every scene stays in one fold. A model that outlines the tampered area, trained on CASIA, does not work well on Columbia (pixel F1 0.43). We compare the results with the five base papers and state this weakness.

Keywords: image forgery detection, splicing, error level analysis, EfficientNet, digital evidence, multi-agent system, web application.

CHAPTER 1

1. INTRODUCTION

1.1 Background

Photographs are accepted as evidence in newsrooms, insurance claims, banks and courts. The same tools that fix a holiday photo can move an object into a scene, remove a person or paste a face onto another body. The result often looks real to the eye. Image forensics asks whether an image is authentic. Passive methods use only the image. They look for traces that editing leaves behind, such as different noise, colour patterns or JPEG compression history between the pasted part and the rest of the picture.

A second question matters just as much. After an image is analysed, can anyone prove which file was checked, what the verdict was and that the record was not changed? A detector that only prints a score cannot answer this. So we built the analysis and the evidence record as one system. Each run gives a verdict, the evidence behind it such as scores, heatmaps and reasons, and a record that anyone can check with cryptography.

1.2 Motivations

- Published splicing detectors report accuracies above 90% on CASIA v2.0 and Columbia, but most are research code. They give a label for one dataset and stop. An analyst needs an application that takes any image and explains its answer.
- A wrong automatic verdict can blame an innocent photographer or clear a fake. The system should be able to say Inconclusive and pass the image to a person.
- Evidence must hold up when it is questioned. A SHA-256 hash of the exact file, a digital signature on the record and an optional public timestamp let a third party check that nothing changed after the analysis.
- No single signal can be trusted alone. Metadata is easy to remove, recompression hides ELA traces and most images carry no provenance data. Several agents cover for each other.

1.3 Scope of the Project

TrustSphere AI analyses still images only. Earlier versions of the project also handled audio and documents. We removed both, and none of their code, data or results appear in this report.

- **Included:** image upload, a metadata agent, an image forensics agent with a trained detector, a provenance agent, a decision engine, SHA-256 hashing, Ed25519 signing, a chain of custody, optional blockchain anchoring, an evidence PDF, a REST API and a web application.
- **Datasets:** CASIA v2.0 and Columbia, the two image datasets used by the base papers.
- **Experiments:** detection accuracy by cross-validation, a test of the full pipeline on held out images and a localisation experiment.
- **Not included:** video, audio, document forensics, training on fully AI generated images and any legal judgement. The score is meant to support a decision, not replace it.

CHAPTER 2

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

Image forgery has three common kinds. In splicing, a part of one image is pasted into another. In copy-move, a part is copied within the same image. In removal, an object is erased and the gap is filled [5]. Classic detectors look for mismatches in noise, in colour filter patterns, in lighting and in double JPEG compression, because a pasted part keeps the statistics of its source image [5]. Error level analysis (ELA) was described for forensic use by Krawetz [10]. It saves an image again as JPEG at a known quality and shows the difference, so parts with a different compression history stand out.

Deep learning has replaced hand made features in most recent work. Salloum et al. [6] trained a fully convolutional network on CASIA v2.0 to predict both the spliced region and its boundary. They tested it on CASIA v1.0, Columbia and other sets. Training on CASIA and testing elsewhere was later used by [5].

Early passive methods use traces left by the camera and the editing steps. Scaling or rotating leaves repeating patterns between neighbouring pixels [15]. Camera colour interpolation leaves patterns that a pasted part tends to lack [16]. Sensor noise works like a fingerprint of the camera, and a region without it may be tampered [17]. Mahdian and Saic look for regions with a different noise level [18]. For JPEG images, JPEG ghosts show a region first saved at another quality [19], and Bianchi and Piva estimate the chance of tampering for each 8×8 block [20]. Copy-move forgery was first found by matching blocks [21]. SIFT keypoints can also give the transformation of the copied region [22], and Christlein et al. compared fifteen feature sets in one pipeline [23]. Splicebuster models noise residuals and splits the image into real and spliced parts without any forged training examples [24]. The filters of the rich models by Fridrich and Kodovský were made for steganalysis [25]. We use them as the SRM input for Columbia.

Deep networks learn these traces from data. Bayar and Stamm add a constrained first layer that suppresses image content and learns features for manipulation detection [26]. Zhou et al. use an RGB stream and a noise stream in a Faster R-CNN [27]. Huh et al. train only on real photos and use their EXIF data to learn whether an image is self consistent, meaning that it could come from one imaging pipeline. They then use this to find splices without ever seeing forged examples [28]. BusterNet separates the source and target regions of copy-move forgery [29]. ManTra-Net learns many manipulation types and treats localisation as finding anomalies [30]. RRU-Net adds residual feedback to a U-Net [31]. PSCC-Net predicts masks step by step at several scales [32]. MVSS-Net learns from the noise view and from boundary artifacts, and is supervised at several scales [33]. CAT-Net learns from JPEG DCT coefficients and pixels [34]. ObjectFormer uses RGB and high frequency features in a transformer [35]. TruFor adds a learned noise fingerprint and a reliability map [36]. Most of these papers are judged on how well they locate the tampered pixels. Our detector answers the image level question of authentic or tampered. The base papers [1] to [4] answer the same question.

Surveys are given in [37] to [40]. Verdoliva covers media forensics and deepfakes [37]. Qureshi and Deriche list pixel based blind methods [38]. Stamm et al. describe the first ten years of information forensics [39]. Warif et al. review copy-move detection [40]. Muhammad et al. tell

authentic from tampered images using a steerable pyramid transform and local binary patterns on the colour channels [41]. Besides CASIA [7] and the Columbia splicing set [8], other datasets in this field are the Columbia set of authentic and spliced blocks [42], CoMoFoD [43], COVERAGE [44] and the Dresden image database [45]. Our building blocks are standard. Residual networks [46] and vision transformers [47] are common alternatives to our EfficientNet backbone [9]. The localiser follows the U-Net layout [48] and training uses AdamW [49]. For evidence, Hasan and Salah used Ethereum smart contracts to track where a media file came from using file hashes [50]. We store a SHA-256 hash of the analysed image in a similar way and add a signature and a chain of custody.

The five base papers of this project are summarised next.

Base paper 1: local optimal oriented pattern with deep features and an SVM [1]

Ranjan et al. extract texture maps with the Local Optimal Oriented Pattern (LOOP), refine them with a pretrained ResNet-50 and classify with an SVM. They test on the Columbia splicing dataset, CASIA v2.0 and a part of the Dresden image database using stratified 5-fold cross-validation. Their average accuracy is 95.20%. Their review table lists earlier methods that use Columbia, with accuracies between about 91% and 93.4%.

Base paper 2: GAN-ViT-CMFD [2]

Ravula and Singh detect and classify copy-move forgery using generative adversarial networks, a vision transformer and spectral clustering together. In their Table 4 they report accuracy of 98.95% on CoMoFoD, 95.2% on MICC-F220 and 94.4% on CASIA v2. Their abstract gives 99.02% accuracy for CoMoFoD.

Base paper 3: deep CNN features with a machine-learning classifier [3]

Rehman and Jain take patch features from a pretrained CNN, fuse them and classify with an SVM. On CASIA v2.0 they report 93.57% accuracy without augmentation and 98.91% with augmentation. Their Table 3 uses 10-fold cross-validation. For batch size 128 it gives accuracy 98.91%, mean 93.39% and standard deviation 8.75%. For batch size 200 it gives accuracy 93.57%, mean 90.25% and standard deviation 7.98%.

Base paper 4: Satyachitra [4]

Deb and Kar propose a CNN based copy-move method with preprocessing, feature extraction, segmentation and localisation steps. They also study how different the datasets are. On CASIA v2.0 they report precision 87.31, recall 90.15 and F1 88.33.

Base paper 5: GIFT, a green image forgery tracker [5]

Chen et al. take features from a pretrained EfficientNet-B4 and train XGBoost models at several levels for the splicing mask. A second version also predicts the mask boundary. No backpropagation is needed. Following [6], they train on CASIA v2.0 and test on CASIA v1.0 and Columbia. They report pixel F1 of 0.5728 (0.6210 with edges) on CASIA v1.0 and 0.6497 (0.6558) on Columbia, with 18.4 million parameters for GIFT.

Table 2.1: Summary of the base papers

Ref.	Method	Datasets	Task	Reported result
[1]	Local pattern + ResNet-50 + SVM	Columbia, CASIA v2.0, Dresden	Image-level detection	95.20% average accuracy
[2]	GAN + ViT + spectral clustering	CoMoFoD, MICC-F220, CASIA v2	Copy-move detection	94.4% on CASIA v2

[3]	CNN patch features + SVM	CASIA v2.0	Copy-move detection	93.57% (98.91% with augmentation)
[4]	CNN copy-move framework	CASIA v2.0, CoMoFoD, MICC-F220, CMFD	Copy-move detection	F1 88.33 on CASIA v2.0
[5]	EfficientNet-B4 + XGBoost	CASIA v2.0 (train); CASIA v1.0, Columbia (test)	Pixel-level localisation	F1 0.6497 on Columbia

2.2 Research Gap

- **Tests that may reward memory.** CASIA v2.0 has tampered images made from authentic images in the same set. A random split can put a tampered image in the test set and its source in the training set. The base papers report random split or cross-validation accuracy and do not say if sources were kept apart. We report both a random split and a split that keeps sources together.
- **Detection without evidence.** The base papers stop at a label or a mask. None of them hashes the file, signs the result or gives a record that a third party can check.
- **Prototypes instead of applications.** The published methods are scripts for one benchmark. An analyst gets no upload page, no explanation of the verdict and no way to review earlier cases.
- **Weak transfer.** The protocol of [5] and [6] trains on CASIA v2.0 and tests on Columbia. It gives pixel F1 of about 0.65 at best. Behaviour on a new dataset is still an open problem, and a system should say so.

2.3 Objectives

1. Build a multi-agent pipeline with metadata, image forensics and provenance agents and a decision engine that returns Authentic, Fake or Inconclusive with reasons.
2. Train an image manipulation detector on CASIA v2.0 and Columbia and measure it with cross-validation, including a split that keeps sources together.
3. Make every analysis tamper evident with a SHA-256 hash, an Ed25519 signature, a chain of custody, an optional blockchain anchor and an evidence PDF.
4. Build a web application and REST API that run on one machine.
5. Compare the results with the five base papers on the same datasets and say where the system falls short.

2.4 Problem Statement

Given a still image of unknown origin, decide whether it is authentic or manipulated and show the evidence for the decision. Say Inconclusive when the evidence is weak. Keep a record of the analysis that a third party can check without trusting the operator.

2.5 Project Plan

The plan follows the history of the project repository. The dates are the dates of the work. The training and evaluation work of October 2026 was done in this semester.

Table 2.2: Project plan

Phase	Activity	Period
1	Project set-up: FastAPI backend, Next.js scaffold, repository	1 July 2026
2 to 5	Backend core, forensic agents, decision engine, security layer (signing,	7 July 2026

	C2PA, blockchain anchor)	
6	Explainability and the evidence PDF, plus an optional local LLM summary that is off in this report	7 to 10 July 2026
7	Scope reduced (video removed 15 July); later reduced to images only	15 July and 24 October 2026
8	Web application: analysis, report, history and how-it-works pages	October 2026
9	Dataset preparation, detector training, cross-validation, hold-out evaluation, localisation experiment	October 2026
10	End-to-end testing, documentation, report	October 2026
Future	Cross-dataset performance, JPEG-compressed images, hosted deployment	After submission

CHAPTER 3

3. TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional

- FR1. Accept an uploaded image (JPEG, PNG, WebP, GIF, BMP or TIFF) up to 100 MB and reject every other file type with HTTP 415.
- FR2. Compute the SHA-256 hash of the exact bytes and keep the original file as evidence.
- FR3. Run the metadata, image forensics and provenance agents and store what each agent finds.
- FR4. Combine the agent scores into an authenticity score between 0 and 1 and a verdict. Authentic is at or above 0.75, Fake is at or below 0.40 and anything between is Inconclusive.
- FR5. Show the reasons, the heatmaps (error level analysis and Grad-CAM) and how much each metadata field contributed.
- FR6. Sign every record with Ed25519, keep a chain of custody and, if a wallet is set, anchor the verdict on the Polygon Amoy blockchain.
- FR7. Create an evidence PDF and let anyone check the file hash, the signature and the anchor again.
- FR8. Provide a web application and a documented REST API.

3.1.2 Non-Functional

- **Accuracy:** only cross-validated and held out figures are reported, with the protocol stated.
- **Speed:** analyse one image in seconds on one GPU. On the 588 held out images the whole pipeline took 0.24 s per image on average, about 142 s in total.
- **Integrity:** any change to a stored file or record must be detectable.
- **Portability:** run on one Windows machine with SQLite and no Redis, and also with PostgreSQL and Redis.
- **Security:** signing keys, wallet keys and environment files are never committed to the repository.
- **Maintainability:** each agent is one module with the same interface, so one agent can be replaced without touching the others.

3.2 Feasibility Study

3.2.1 Technical Feasibility

Every part is available as open source software (Python, PyTorch, timm, FastAPI, SQLAlchemy, ReportLab and web3). The detector is small. EfficientNet-B0 has about 4 million parameters, and we trained and tested it on one laptop GPU with 8 GB of memory. We start from pretrained ImageNet weights, so no large dataset or cluster is needed.

3.2.2 Economic Feasibility

No paid service was used. The models and libraries are open source and the datasets are public. The database is SQLite or a free PostgreSQL plan. Blockchain anchoring uses the Polygon Amoy

test network, where gas is free. The only hardware is the development laptop, so the running cost is only electricity.

3.2.3 Social Feasibility

The system helps journalists, insurers and investigators sort images. The main risk is trusting an automatic verdict too much. The design reduces this with an Inconclusive band, visible reasons and heatmaps, and a note that scores are not a legal finding. Images are processed on the operator's own machine and are not sent to anyone else.

3.3 System Specification

3.3.1 Hardware Specification

Table 3.1: Hardware used for development, training and testing

Component	Specification
Computer	Laptop, Windows 11
GPU	NVIDIA GeForce RTX 3070 Ti Laptop GPU, 8 GB
Memory	About 16 GB RAM
Storage	Local SSD

3.3.2 Software Specification

Table 3.2: Software stack

Layer	Software
Language	Python 3.13
Deep learning	PyTorch 2.6 (CUDA 12.4), torchvision 0.21, timm 1.0
Image and data	Pillow, NumPy, pandas, scikit-learn, OpenCV, matplotlib
Explainability	Grad-CAM (pytorch-grad-cam), closed-form Shapley values for metadata
Backend	FastAPI, Uvicorn, SQLAlchemy, Celery (optional)
Database	SQLite (local mode) or PostgreSQL
Security and provenance	cryptography (Ed25519), c2pa-python, invisible-watermark, web3 (Polygon Amoy)
Reporting	ReportLab (evidence PDF)
Web app	HTML, CSS and JavaScript served by FastAPI (no external libraries)
Browser	Any current browser

CHAPTER 4

4. DESIGN APPROACH AND DETAILS

4.1 System Architecture

The user uploads an image in the web application. The FastAPI backend stores the file, hashes it and runs three agents. Each agent returns a score from 0 (fake) to 1 (authentic) and a list of findings. The decision engine takes the weighted average of the scores. The weights are 0.30 for image forensics, 0.25 for provenance and 0.20 for metadata. An agent with no signal is left out instead of being counted as 0.5. The average becomes a verdict. The record is then signed, saved in the database with a chain of custody, anchored on the blockchain if set up, and made available as JSON and as an evidence PDF.

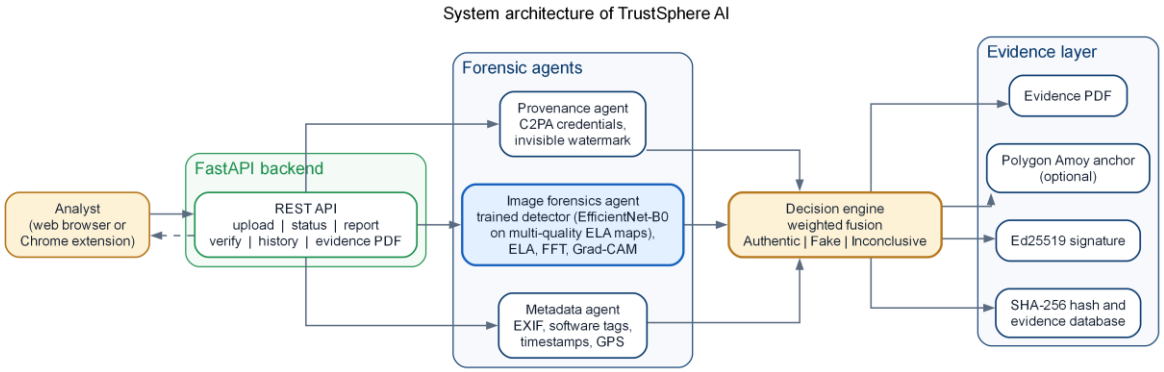


Figure 4.1: System architecture of TrustSphere AI

4.2 Design

4.2.1 Data Flow Diagram

Figure 4.2 shows the level 1 data flow. The original file and its hash go to the evidence store. The forensic agents read the trained model weights. The signed record goes back to the store and the report goes to the analyst.

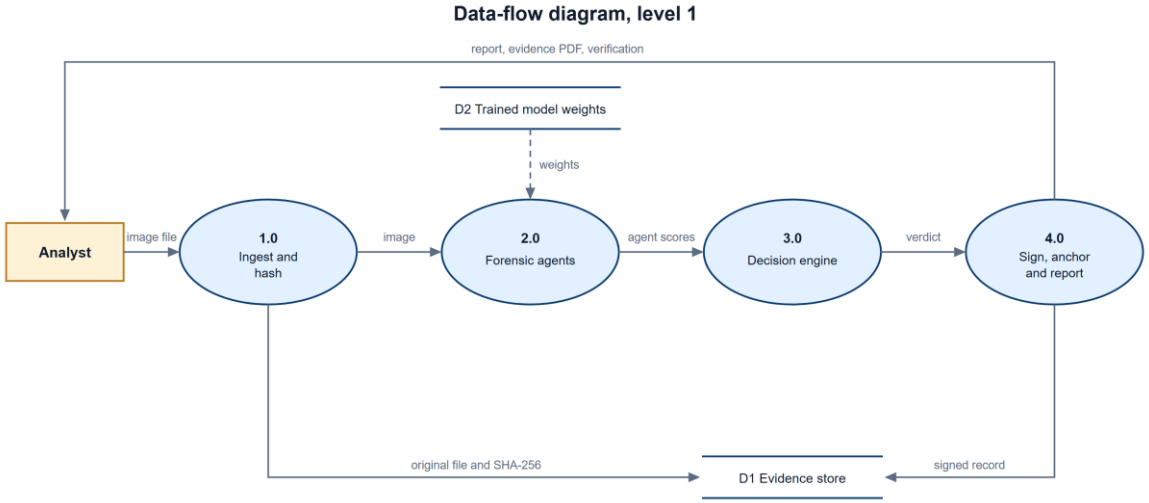


Figure 4.2: Data-flow diagram, level 1

4.2.2 Use Case Diagram

The analyst uploads images, reads verdicts and heatmaps, downloads the evidence PDF and browses the history. A verifier or auditor does not have to trust the analyst, because the verdict, the evidence PDF and the integrity check are open to anyone who has the file identifier.

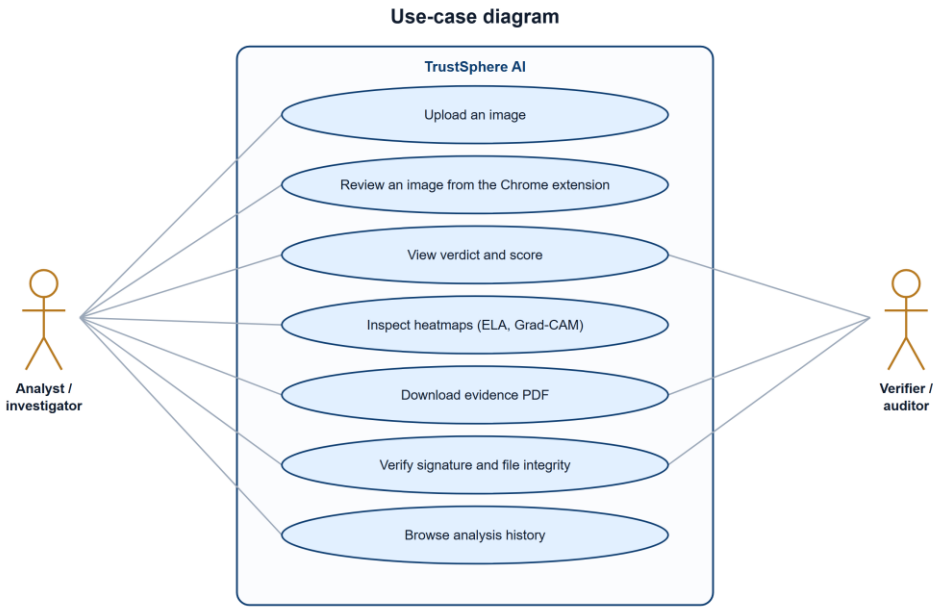


Figure 4.3: Use-case diagram

4.2.3 Class Diagram

The data model has four tables. Evidence holds the uploaded file, AgentResult has one row per agent, FinalVerdict has one row per analysis and AuditLog holds the signature, anchor and chain of custody. All agents use the same run(path) interface, so the decision engine treats them the same way.

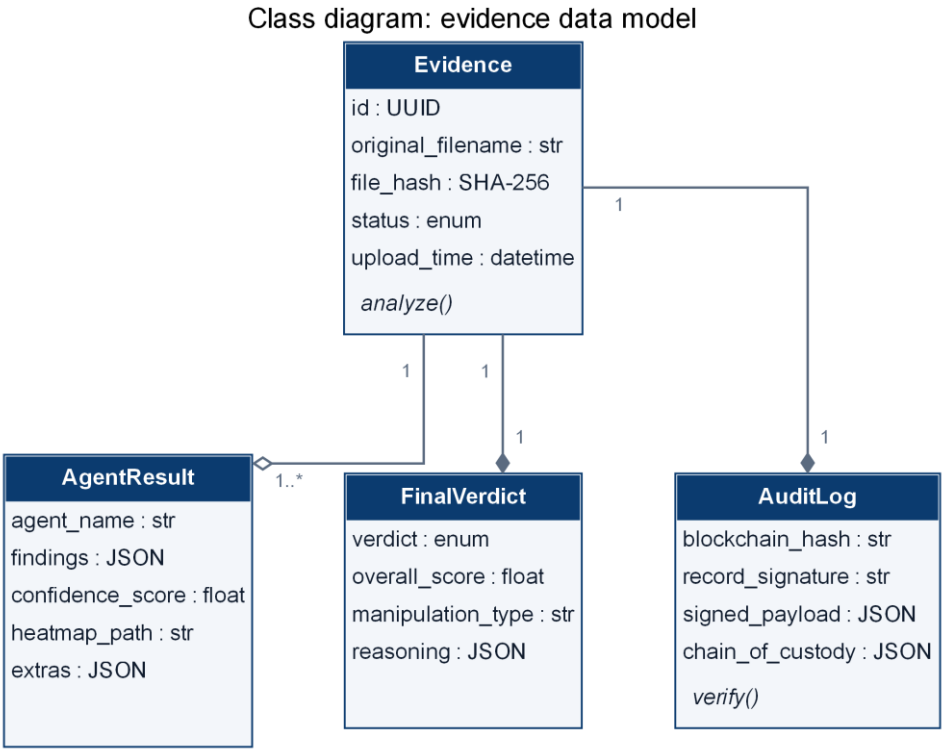


Figure 4.4: Class diagram: evidence data model

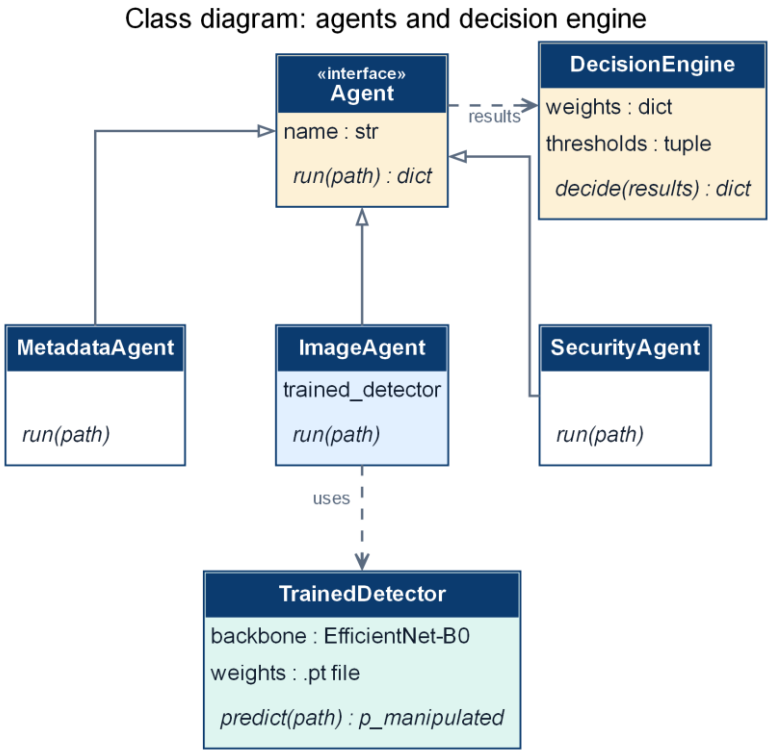


Figure 4.5: Class diagram: agents and decision engine

4.2.4 Sequence Diagram

Figure 4.6 shows one analysis from upload to report. Report and verification are separate requests,

so verification can be repeated at any later time.

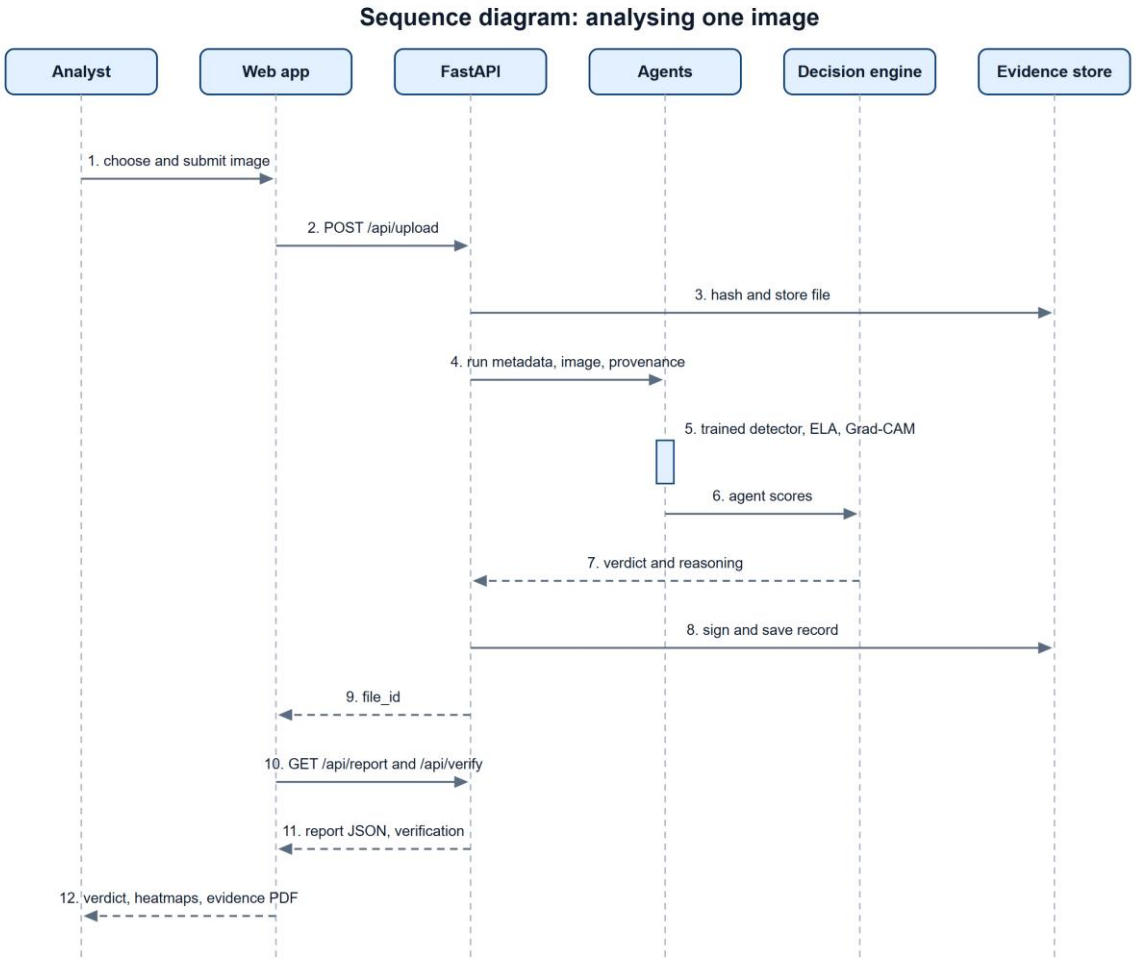


Figure 4.6: Sequence diagram: analysing one image

CHAPTER 5

5. METHODOLOGY AND TESTING

5.1 Module Description

5.1.1 Ingestion and evidence store

The upload endpoint accepts only image MIME types. The file is saved under its SHA-256 hash, a database row is created and the analysis is queued as a Celery task. With Redis the task runs in a worker process. In local mode, without Redis, it runs inline and the upload call returns when the analysis is done. The status endpoint reports pending, analyzing, done or failed, and the web application polls it in both modes.

5.1.2 Metadata agent

The metadata agent reads EXIF data and flags a missing camera make and model, editing or generator software in the tags, missing or inconsistent timestamps and GPS data. Each check gives a score. A closed form Shapley value shows how much each check moved the score towards authentic or fake. Images with no EXIF at all, such as the PNG files in the datasets, score about 0.45 on this agent.

5.1.3 Image forensics agent

The agent combines supporting heuristics with a trained detector that decides the verdict.

- **Error level analysis:** the image is saved again as JPEG at quality 90 and the amplified difference is saved as a heatmap.
- **Frequency analysis:** the share of high frequency energy in the Fourier spectrum.
- **Grad-CAM [11]:** the attention of a pretrained deepfake classifier, shown next to the ELA map.
- **Trained manipulation detector (this work):** described below. When it is installed, the three heuristics stay in the report as supporting evidence but do not change the score.

Detector input. The image is saved again as JPEG at quality 70, 85 and 95. For each quality we take the absolute difference from the original, turn it to grey scale, amplify it and keep it as one channel. This gives a three channel error level map at the native resolution of the image (Figure 5.1). We avoid resizing on purpose because it destroys the faint compression traces the detector needs.

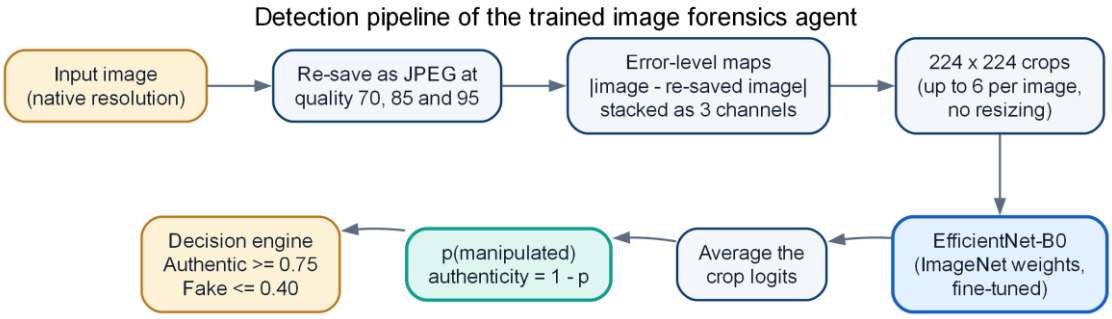


Figure 5.1: Detection pipeline of the trained image forensics agent

Model and training. The network is an EfficientNet-B0 [9] with ImageNet weights and a single

output. Training uses random 224 × 224 crops of the error level maps with random flips. We use the AdamW optimiser with learning rate 4×10^{-4}, a one cycle schedule, weight decay 10^{-4}, batches of 32, mixed precision and 12 epochs. At test time up to six crops that cover the image are scored and their outputs are averaged. The authenticity score is one minus the predicted probability of manipulation.

Choices made on a development split. We chose the input type (one ELA at quality 90 or the three quality map), the backbone (B0 or B3) and the learning rate on one development split of CASIA v2.0. This was a validation part of the training data and the held out test part was not used. Three quality ELA with B0 gave the best validation accuracy and AUC, and we fixed it before any cross-validation. We also tried a six channel input that adds the RGB image to the error maps on one fold. It was clearly worse, so we stopped it and do not use it.

5.1.4 Provenance agent

The provenance agent checks for embedded C2PA content credentials [14] and for invisible watermarks written by AI image generators. A valid manifest moves an image straight to Authentic. A failed manifest or a generator watermark is reported as suspicious and appears in the reasons. Most images have neither. In that case the agent reports no signal and is left out of the weighted average.

5.1.5 Decision engine

The engine takes the weighted mean of the agent scores and applies the thresholds. Authentic is at or above 0.75, Fake is at or below 0.40 and anything between is Inconclusive. It also writes a list of reasons. When the trained detector flags the image, the manipulation type is shown as Image Splicing / Tampering. The Inconclusive band is deliberate. It sends borderline images to a person.

5.1.6 Evidence layer

The analysed file is hashed with SHA-256. A standard form of the record (file hash, verdict, score, agent scores and time) is signed with an Ed25519 key [13] and the fingerprint of the public key is stored with it. The chain of custody lists the upload, analysis, signing and anchoring events. If a wallet is set, the file hash and verdict are written to a smart contract on Polygon Amoy and can be read back for checking. The evidence PDF holds the identification, the verdict and reasons, the findings of each agent, the explanation exhibits, the chain of custody and the signature.

5.1.7 Web application

The web application is a single page served by the backend. The Analyze page uploads an image and shows the stages. The Report page shows the verdict gauge, the reasons, the findings of each agent, the heatmaps, the Shapley chart, the evidence chain with live checking and the chain of custody. The History page lists past analyses. The How it works page explains the agents and the weights.

5.1.8 Pixel-level localisation experiment

To compare with [5], we trained a U-Net style network on the CASIA v2.0 masks. It uses an EfficientNet-B0 encoder on the RGB image plus the three quality ELA map, a bilinear feature pyramid decoder and a loss that adds binary cross-entropy and Dice. We tested it on Columbia. Columbia was never used for training, choosing the threshold or early stopping. The threshold came from a CASIA validation part that keeps sources apart.

5.2 Testing

5.2.1 Evaluation protocols

- **Random 5-fold cross-validation** (stratified, seed 42). This is closest to the base papers.
- **Source-disjoint 5-fold cross-validation** on CASIA v2.0. Every tampered image stays in the same fold as the authentic image it was made from, using the source named in the metadata files of the dataset.
- **Held out test of the full pipeline.** An 80/20 split that keeps sources together. A detector trained on the 80% was used by the real agents and the decision engine on the 588 images of the other 20%.
- **Columbia.** 5-fold cross-validation over its 363 images with several inputs tried. All are reported in Chapter 7.
- All metrics come from out-of-fold or held out predictions, so no image is scored by a model that trained on it.

5.2.2 Functional and end-to-end testing

A script (scripts/e2e_test.py) drives a running server through its public API with six images, three authentic and three tampered. All 19 checks passed. This includes the tamper check. Changing one byte of a stored evidence file makes the verification endpoint report TAMPERED, and restoring the file makes it pass again.

Table 5.1: End-to-end test results

Check	Result	Detail
GET /health	pass	
GET /app/ serves the web app	pass	
GET /app/app.js	pass	
GET /docs (OpenAPI)	pass	
analyse Au_ani_30064.jpg.png	pass	verdict Authentic score 0.780, 71.9 s, pdf 290 KB
analyse Au_ani_30509.jpg.png	pass	verdict Authentic score 0.780, 9.8 s, pdf 244 KB
analyse Au_ani_30704.jpg.png	pass	verdict Authentic score 0.780, 6.7 s, pdf 318 KB
analyse Tp_D_CRN_M_N_ani10105_ani00100_100	pass	verdict Fake score 0.368, 7.0 s, pdf 265 KB
analyse Tp_D_CRN_M_N_art10112_cha00086_116	pass	verdict Fake score 0.183, 8.1 s, pdf 335 KB
analyse Tp_D_NRN_S_N_art00051_art00010_010	pass	verdict Fake score 0.180, 8.8 s, pdf 243 KB
history lists every analysis	pass	6 stored
reject text/plain	pass	HTTP 415
reject audio/wav	pass	HTTP 415
reject application/pdf	pass	HTTP 415
reject video/mp4	pass	HTTP 415
unknown report id -> 404	pass	
unknown verify id -> 404	pass	
altering the stored file is detected	pass	integrity TAMPERED
restoring the file verifies again	pass	integrity intact

CHAPTER 6

6. PROJECT DEMONSTRATION

The screenshots below come from the running application. The images were taken from the held out part of CASIA v2.0 and analysed with the hold-out detector, so the detector had never seen them.

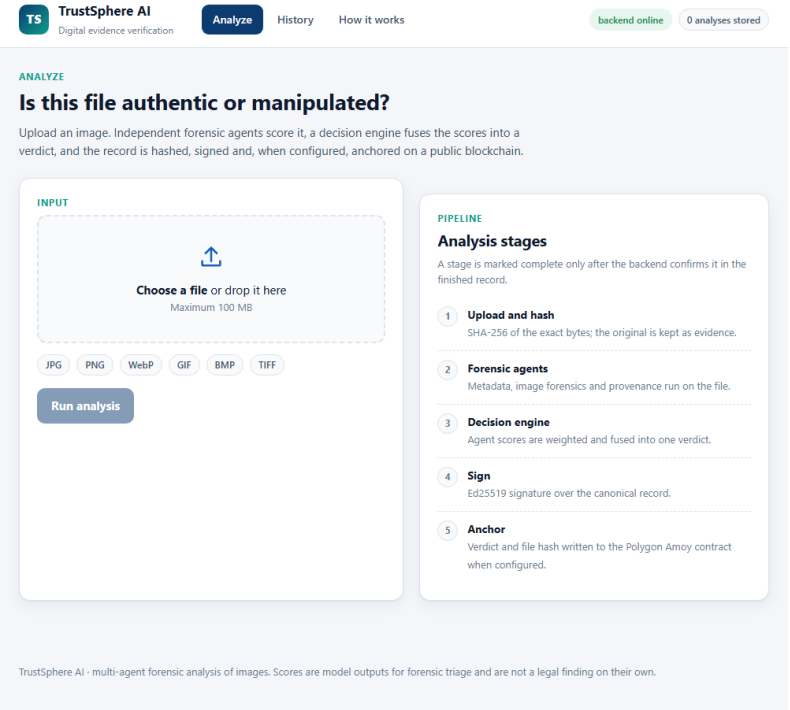


Figure 6.1: Analyze page: the user chooses an image and sees the analysis stages

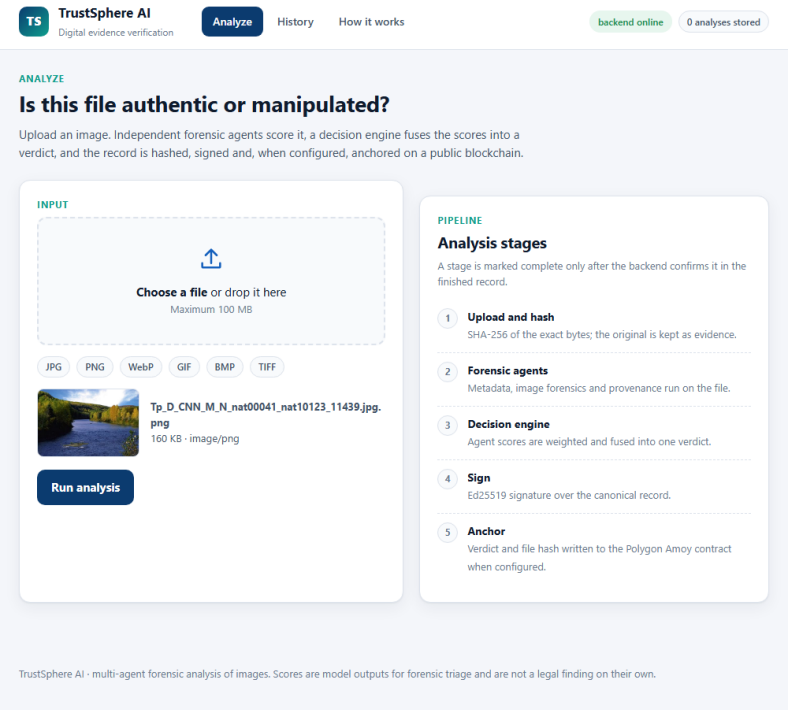


Figure 6.2: A file selected for analysis

For a tampered image the report opens with the verdict. It shows a gauge of the authenticity score,

the manipulation type, the reasons and the file hash (Figure 6.3). The agent breakdown comes next and shows what each agent found. The image forensics card puts the original, the error level map and the Grad-CAM attention map side by side and gives the probability from the trained detector (Figure 6.4). The evidence chain checks the file hash and the signature again on every visit and lists the chain of custody (Figure 6.5).

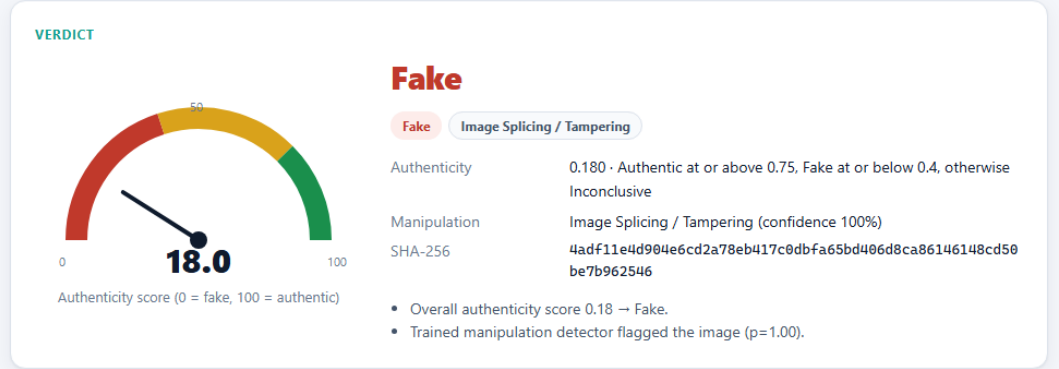


Figure 6.3: Verdict card for a tampered image



Figure 6.4: Agent breakdown for the tampered image: metadata, image forensics with heatmaps, provenance

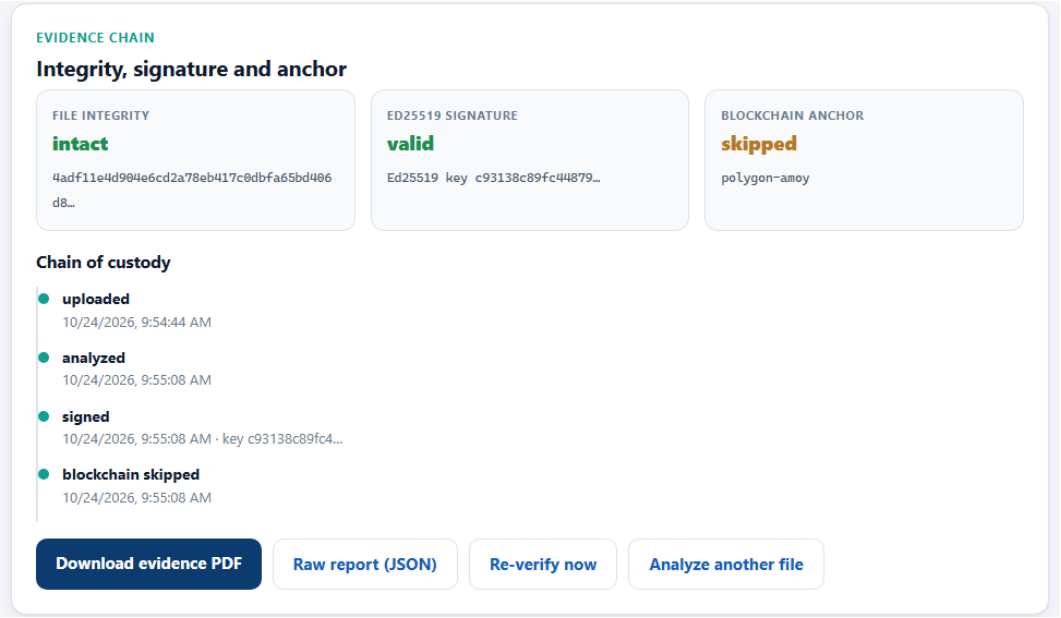


Figure 6.5: Evidence chain: file integrity, Ed25519 signature, anchor status and chain of custody

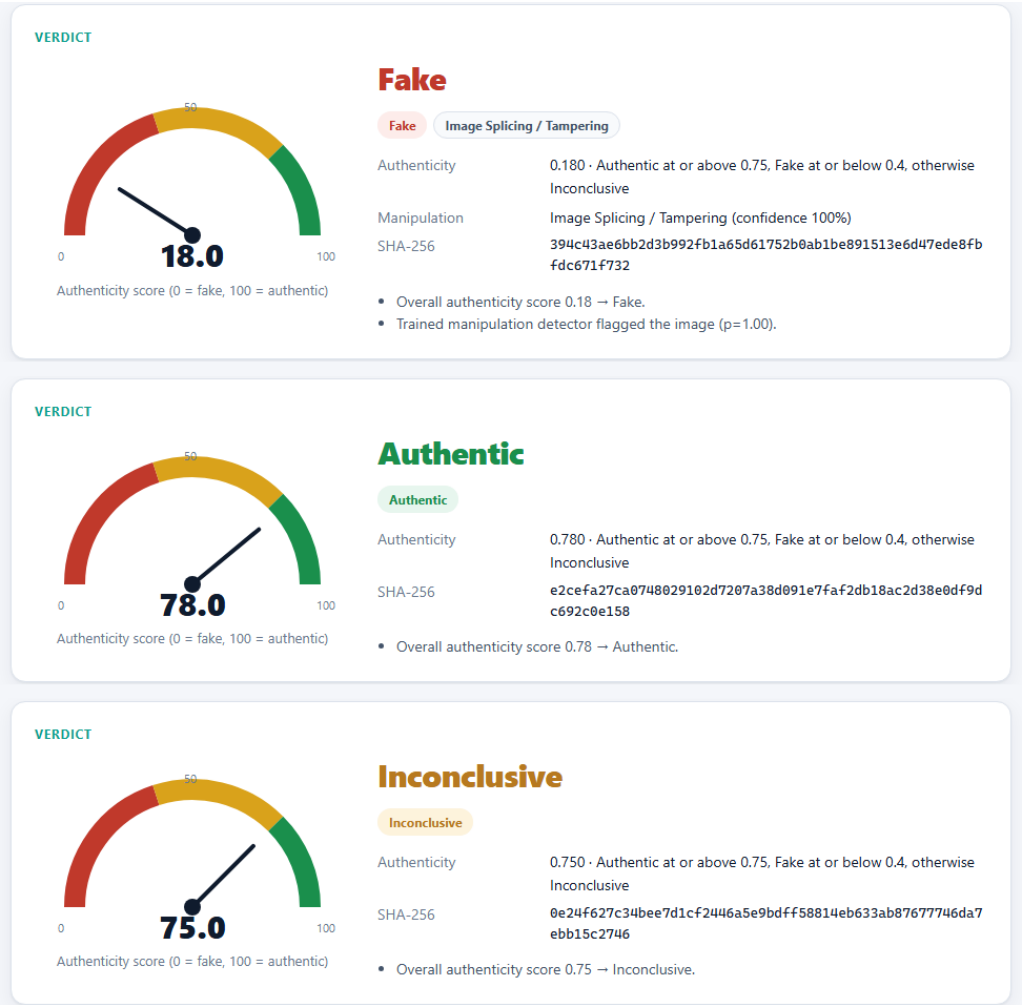


Figure 6.6: Verdict cards of a second tampered image (Fake), an authentic image (Authentic) and an authentic image the engine sends to human review (Inconclusive)

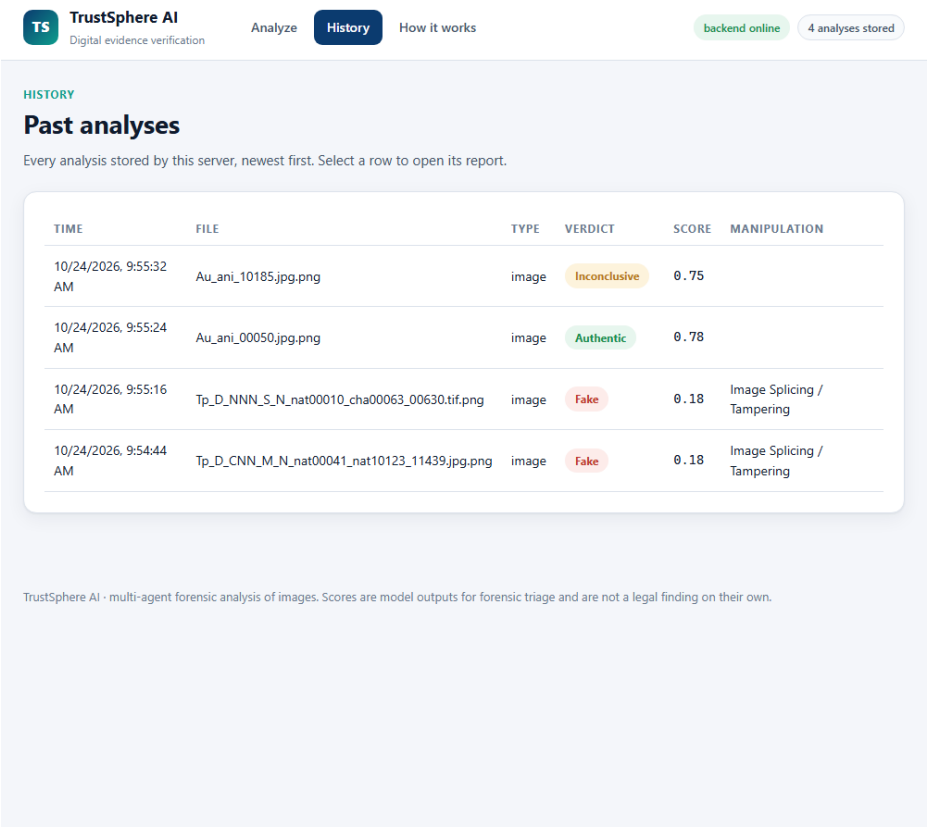


Figure 6.7: History page: every stored analysis with its verdict and score

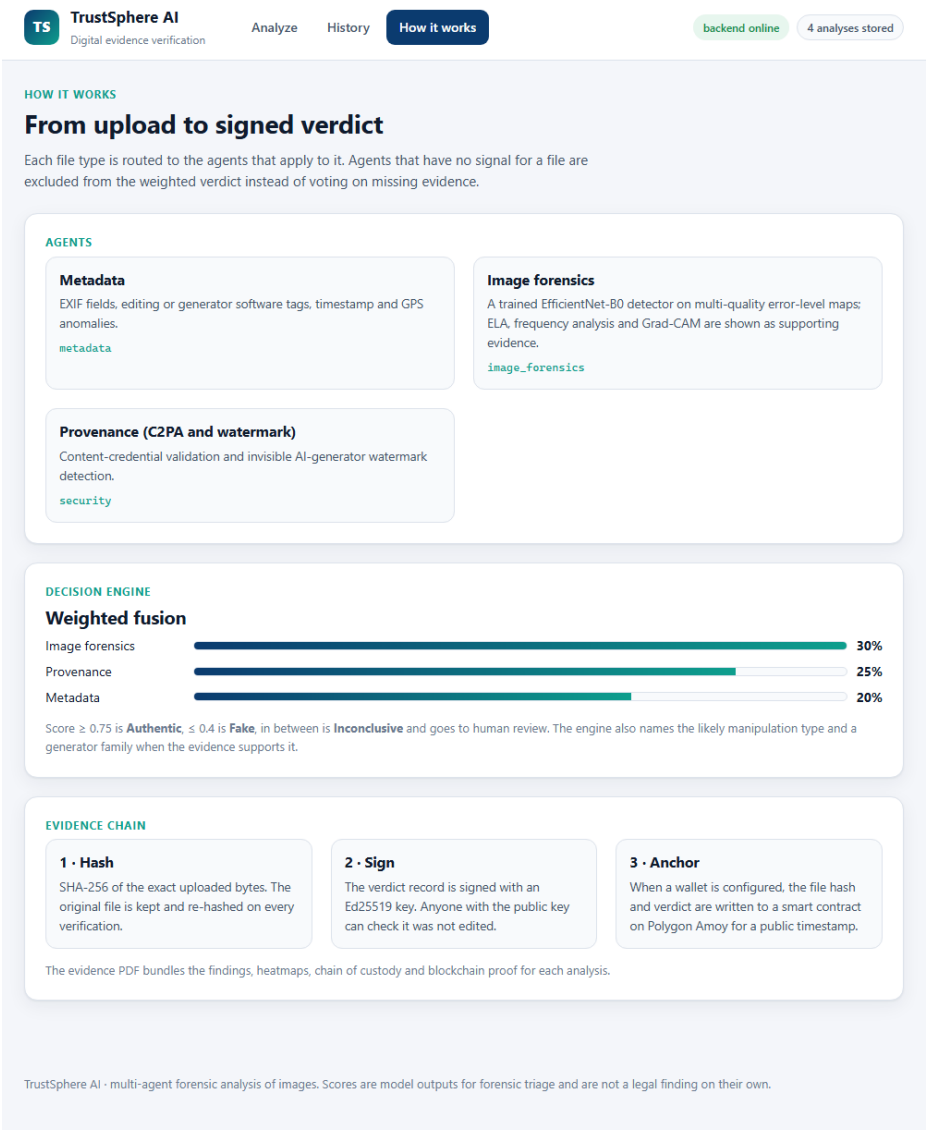
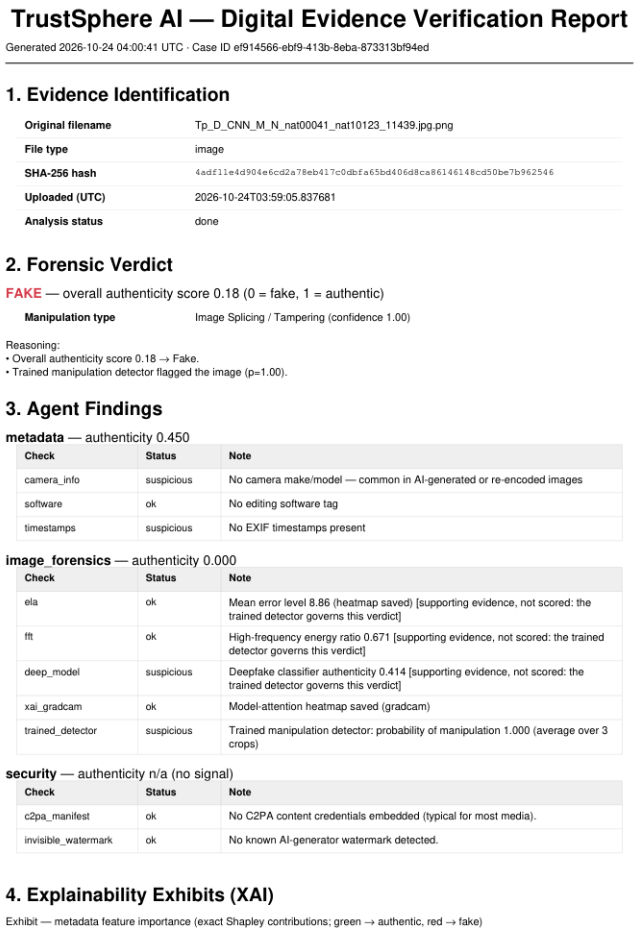


Figure 6.8: How it works page: the agents, the fusion weights and the evidence chain

Each analysis can be downloaded as an evidence PDF with the identification, verdict, findings, heatmaps, chain of custody and signature.



TrustSphere AI — Digital Evidence Verification Report

Generated 2026-10-24 04:00:41 UTC · Case ID ef914566-ebf9-413b-8eba-873313bf94ed

1. Evidence Identification

Original filename	Tp_D_CNN_M_N_nat00041_nat10123_11439.jpg.png
File type	image
SHA-256 hash	4adf11e4d904e6cd2a78eb417c0dbfa65bd406d8ca86146148cd50be7b962546
Uploaded (UTC)	2026-10-24T03:59:05.837681
Analysis status	done

2. Forensic Verdict

FAKE — overall authenticity score 0.18 (0 = fake, 1 = authentic)

Manipulation type	Image Splicing / Tampering (confidence 1.00)

Reasoning:
• Overall authenticity score 0.18 → Fake.
• Trained manipulation detector flagged the image (p=1.00).

3. Agent Findings

metadata — authenticity 0.450

Check	Status	Note
camera_info	suspicious	No camera make/model — common in AI-generated or re-encoded images
software	ok	No editing software tag
timestamps	suspicious	No EXIF timestamps present

image_forensics — authenticity 0.000

Check	Status	Note
ela	ok	Mean error level 8.86 (heatmap saved) [supporting evidence, not scored: the trained detector governs this verdict]
fft	ok	High-frequency energy ratio 0.671 [supporting evidence, not scored: the trained detector governs this verdict]
deep_model	suspicious	Deepfake classifier authenticity 0.414 [supporting evidence, not scored: the trained detector governs this verdict]
xai_gradcam	ok	Model-attention heatmap saved (gradcam)
trained_detector	suspicious	Trained manipulation detector: probability of manipulation 1.000 (average over 3 crops)

security — authenticity n/a (no signal)

Check	Status	Note
c2pa_manifest	ok	No C2PA content credentials embedded (typical for most media).
invisible_watermark	ok	No known AI-generator watermark detected.

4. Explainability Exhibits (XAI)

Exhibit — metadata feature importance (exact Shapley contributions; green → authentic, red → fake)

Figure 6.9: First page of the evidence PDF of a tampered image

CHAPTER 7

7. RESULT AND DISCUSSION

7.1 Experimental Setup

The datasets are a subset of CASIA v2.0 and the Columbia set. Our copy of CASIA v2.0 has 1,536 authentic and 1,283 tampered images, all PNG. The full dataset has 7,491 authentic and 5,123 tampered images, so our results are not on the full set. Columbia has 183 authentic and 180 spliced images. Both come from the Hugging Face collection ductai199x/image-manipulation-dataset-compilation [12]. Label 1 means manipulated. We report accuracy, precision, recall, F1, ROC AUC and the equal error rate (EER). A confusion matrix is shown for each experiment.

7.2 Detector Accuracy on CASIA v2.0

Table 7.1: Detector results on CASIA v2.0

Protocol	Accuracy (%)	Precision	Recall	F1	ROC AUC	EER
Random 5-fold	95.53 ± 0.86	95.16%	95.01%	95.09%	0.9917	4.58%
Source-disjoint 5-fold	95.32 ± 0.80	95.28%	94.39%	94.83%	0.9933	4.47%
Hold-out 20% (588 images)	96.26%	96.49%	95.82%	96.15%	0.9943	3.23%

The two cross-validation protocols give almost the same accuracy, 95.53% and 95.32%. Keeping each tampered image with its source did not change the result much. This only controls for the source named in the dataset metadata and not for every near duplicate. The fold accuracies were 95.9, 95.4, 96.8, 94.2 and 95.4 percent for the random split, and 94.0, 96.3, 95.4, 95.9 and 95.0 percent when sources were kept together.

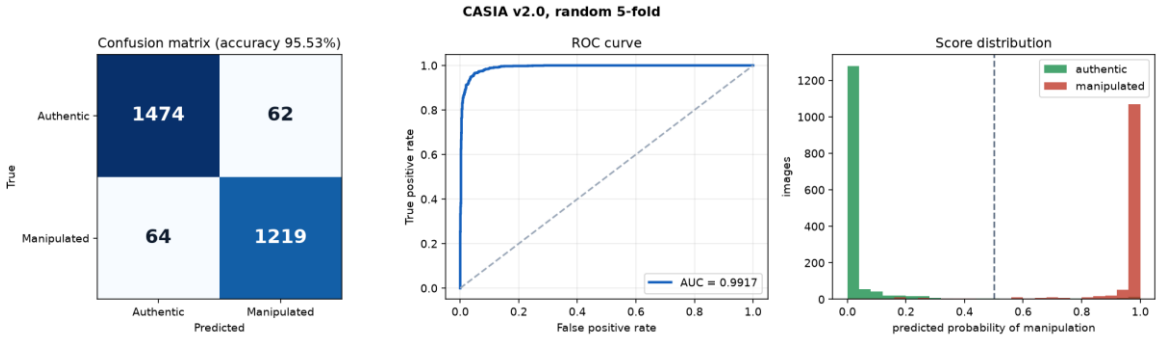


Figure 7.1: CASIA v2.0, random 5-fold cross-validation: confusion matrix, ROC curve and score distribution

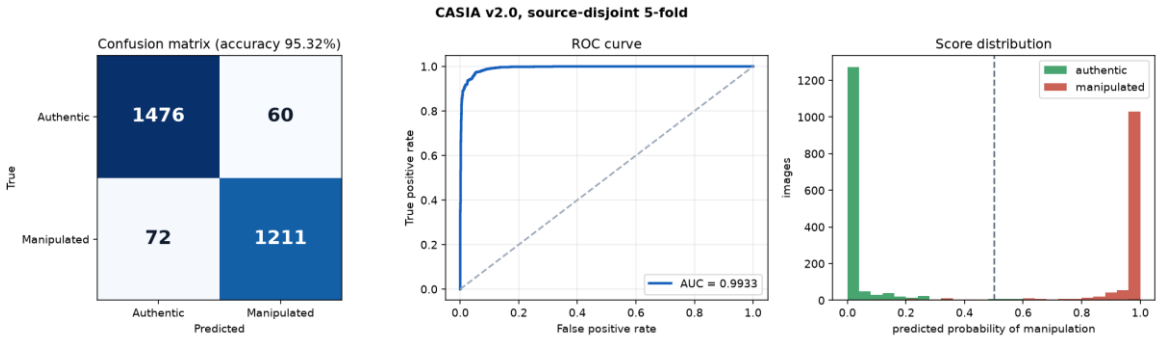


Figure 7.2: CASIA v2.0, source-disjoint 5-fold cross-validation

7.3 The Complete Pipeline on Held-Out Images

The real agents and the decision engine ran on the 588 held out images (301 authentic and 287

tampered). The engine gave a verdict on 90.5% of them and was right on 97.9% of those. The rest went to review. If Inconclusive counts as a miss, the strict accuracy is 88.6%. If the engine had to choose using the 0.5 mark of its score, it would reach 96.43% (AUC 0.9943).

Table 7.2: Decision-engine verdicts on the held-out images

	Authentic	Fake	Inconclusive
True authentic	251	8	42
True tampered	3	270	14

Authentic images were called Fake in 2.7% of cases and tampered images were called Authentic in 1.0%. Most of the cost of the Inconclusive band falls on authentic images. The metadata agent gives a PNG with no EXIF about 0.45, which pulls the combined score towards the middle.

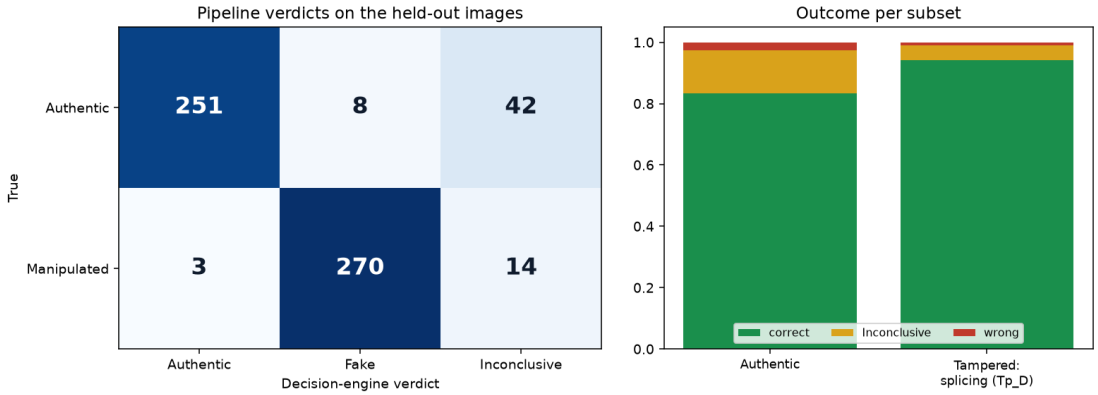


Figure 7.3: Pipeline verdicts on the held-out images

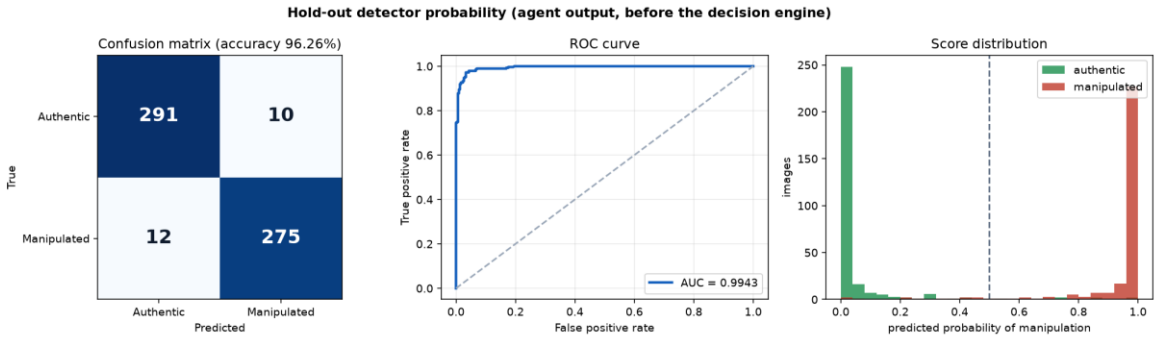


Figure 7.4: Hold-out detector probability before the decision engine

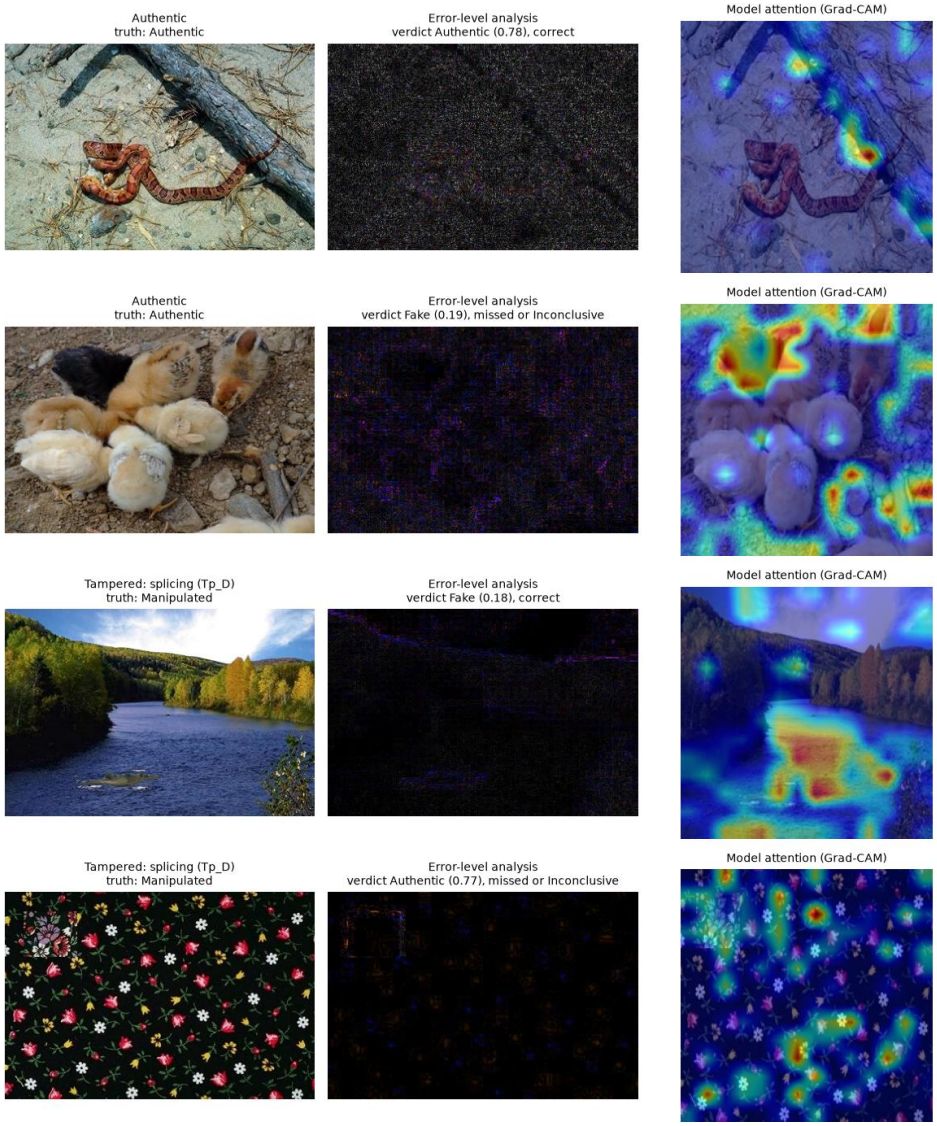


Figure 7.5: Sample outputs from the held-out set: input, error-level analysis and Grad-CAM

7.4 Columbia

Table 7.3: Columbia, 5-fold cross-validation (all variants tried are listed)

Method	Accuracy (%)	Precision	Recall	F1	ROC AUC	EER
Whole image, ELA input	87.60 ± 4.27	84.26%	92.22%	88.06%	0.9610	11.85%
Whole image, SRM noise-residual input	86.49 ± 5.12	81.95%	93.33%	87.27%	0.9490	14.88%
Patch based, SRM input	97.25 ± 2.12	96.70%	97.78%	97.24%	0.9872	2.48%
Patch based, SRM input, scenes kept apart	96.92 ± 2.43	96.69%	97.22%	96.95%	0.9799	2.76%

Columbia has uncompressed photographs, so error level analysis, which depends on JPEG history, has little to work with. Also, 363 images are few for training a network on whole images. We tried three variants and all are listed above. The whole image variants are much less stable than on CASIA, and their folds differ by more than ten points. The patch based variant cuts the image into 128 pixel patches and labels them using the ground truth masks. It scores an image by the mean of its top 5% patch probabilities and takes the decision threshold from the other folds. Columbia photographs share scenes, so the last row repeats the test with every scene kept inside one fold. It

is the stricter figure.

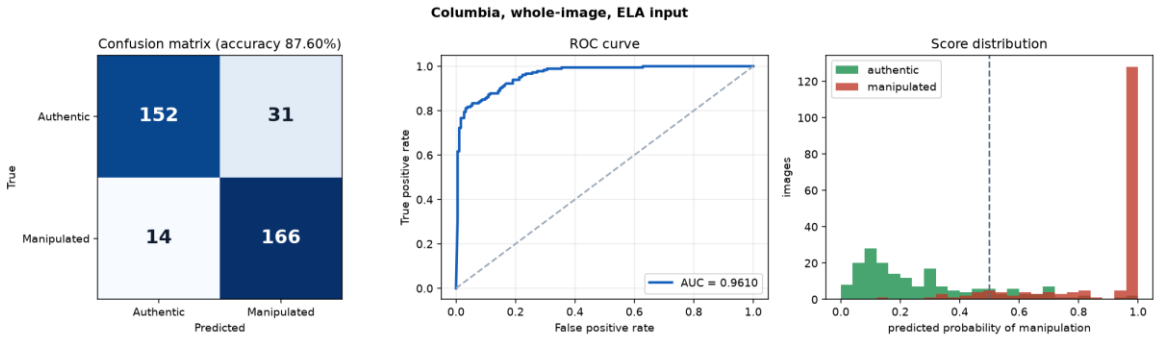


Figure 7.6: Columbia, whole-image detector with ELA input

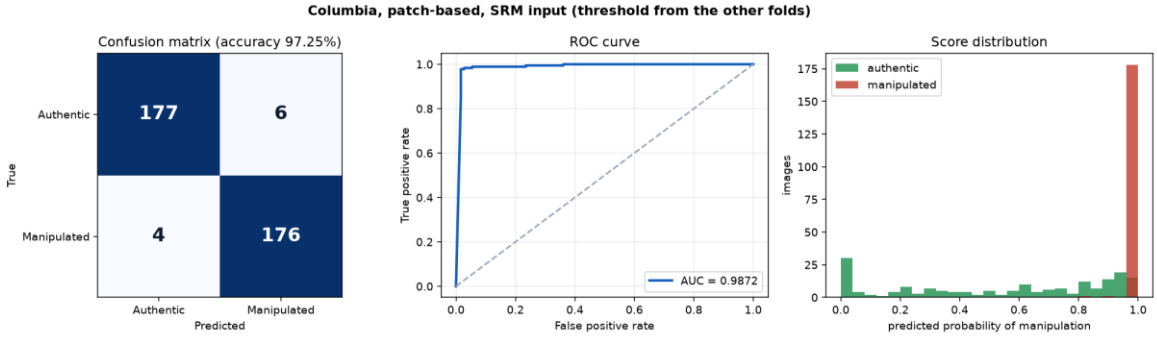


Figure 7.7: Columbia, patch-based detector with SRM input

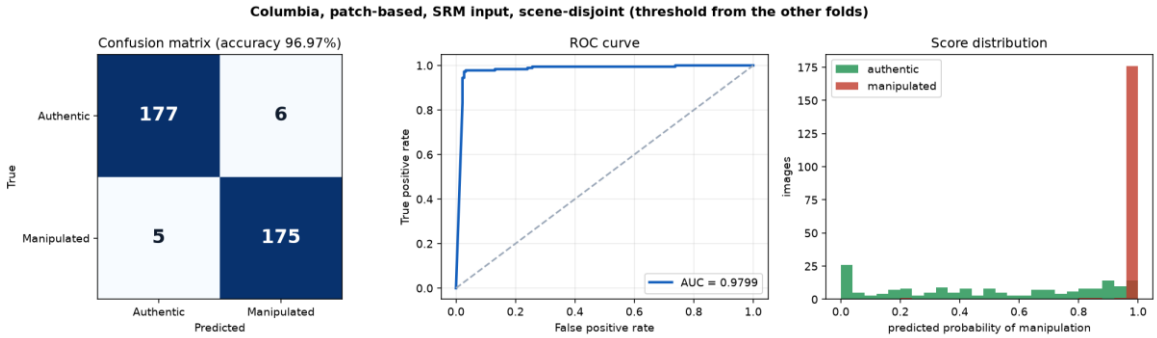


Figure 7.8: Columbia, patch-based detector with scenes kept apart

7.5 Pixel-Level Localisation

Table 7.4: Localiser trained on CASIA v2.0

Test set	Pooled pixel F1	Mean per-image F1	Pixel AUC	Image AUC from the map
CASIA v2.0 validation (source-disjoint)	0.8889	0.7554	0.9569	0.9868
Columbia (never used in training)	0.4330	0.3885	0.6743	0.7994

The localiser finds the tampered region well on images like its training data (pooled F1 0.89) but does not carry over to Columbia (pooled F1 0.43, and 0.48 at the best possible threshold). Localisation across datasets is a limit of this work. The verdict does not use it.

7.6 Comparison with the Base Papers

Only the base paper results that our detector exceeds are listed, using a comparable metric on the same dataset. The protocols differ in splits, subsets and training data. For example, some papers use the full CASIA v2.0 and we use a subset of it. So the table gives context and is not a ranking.

The full comparison, including results we do not reach, is in the file docs/evaluation/EVALUATION.md in the repository.

Table 7.5: CASIA v2.0 results of the base papers that the trained detector exceeds

Base paper	Metric	Reported	TrustSphere (random / source-disjoint)
[2] Ravula and Singh, 2025	Accuracy	94.4%	95.53% / 95.32%
[3] Rehman and Jain, 2025	Mean accuracy over folds	93.39% (best single figure 98.91%)	95.53% / 95.32%
[4] Deb and Kar, 2026	Precision / recall / F1	87.31 / 90.15 / 88.33	95.16 / 95.01 / 95.09 (random)

Table 7.6: Columbia results of earlier methods listed in [1] that the patch-based detector exceeds

Source	Metric	Reported	TrustSphere (random / scenes kept apart)
Earlier Columbia methods in the review table of [1]	Accuracy	91% to 93.4%	97.25% / 96.97%

7.7 Discussion and Limitations

- The ELA based detector works well on CASIA v2.0, where most images come from JPEG files, and poorly on uncompressed Columbia images. The patch based detector on noise residuals fixes this on Columbia but it is a separate model. The deployed application uses the CASIA detector, and a real system will meet both kinds of image.
- Columbia has only 363 images and its photographs share scenes. Keeping each scene in one fold lowers accuracy only a little, but with so few images the spread between folds is still several points. The threshold of the patch based detector comes from the other folds' out-of-fold scores, and those scores come from models that saw the test fold. The AUC does not depend on a threshold, so it is the safer figure.
- CASIA v2.0 has known quirks. Tampered images are made from authentic images in the same set, so even the split that keeps sources together is only partly strict. The held out test uses the same family of data and says little about unrelated images.
- The Inconclusive band costs coverage, since about one image in ten goes to review. This is the price for keeping wrong verdicts at about 3% for authentic images and about 1% for tampered images.
- The localiser does not carry over to other datasets, so heatmaps are only supporting evidence. The verdict does not depend on them.
- All figures are for splicing type tampering in CASIA and Columbia. Fully AI generated images, heavy recompression and screenshots were not tested.

7.8 Cost Analysis

Table 7.7: Cost of building and running the system

Item	Cost
Software, models and libraries	Open-source, no licence cost
Datasets	Public, no cost
Database	SQLite locally; free-tier PostgreSQL optional
Blockchain anchoring	Polygon Amoy test network, free test tokens
Training compute	Existing laptop GPU; a cross-validation run takes tens of minutes
Analysis cost per image	About 0.24 s on the GPU (measured on 588 images)

CHAPTER 8

8. CONCLUSION

TrustSphere AI shows that image forensics and evidence integrity can be built as one system. A multi-agent pipeline analyses an uploaded image and a decision engine returns Authentic, Fake or Inconclusive with its reasons. Every result is hashed, signed, stored with a chain of custody and exported as an evidence PDF. A web application and a REST API sit on top.

The trained detector reaches 95.53% accuracy on CASIA v2.0 with random 5-fold cross-validation and 95.32% when tampered images stay with their sources. On 588 held out images it reaches 96.26%, and the full pipeline gives a verdict on about 90% of them with 97.9% accuracy. These results are higher than those the base papers in Section 7.6 report for CASIA v2.0 under the protocols stated. The work has limits too. A whole image detector is weak on uncompressed Columbia images. A patch based one reaches about 97% on a very small dataset. A localiser trained on CASIA does not carry over to Columbia.

Future work

- Merge the CASIA and Columbia detectors into one model that handles compressed and uncompressed images, and test it on larger uncompressed sets.
- Test on JPEG compressed web images and other benchmarks to measure transfer, and add a warning for images that look unlike the training data.
- Improve localisation across datasets so the heatmap can support the verdict.
- Rebalance the metadata agent so images without EXIF are not pushed to Inconclusive, and tune the verdict thresholds on held out data.
- Host the application and add user accounts and access control for case management.

CHAPTER 9

9. REFERENCES

1. R. Ranjan, D. Nandan, M. K. Singh, S. Kumar, K. V. V. Reddy, V. L. Narla and D. K. Jain, "A novel local optimal oriented pattern for image splicing detection leveraging deep learning and SVM," Franklin Open, vol. 16, p. 100634, 2026, doi: 10.1016/j.fraope.2026.100634.
2. J. Ravula and N. Singh, "GAN-ViT-CMFD: A novel framework integrating generative adversarial networks and vision transformers for enhanced copy-move forgery detection and classification with spectral clustering," Intelligent Systems with Applications, vol. 26, p. 200524, 2025, doi: 10.1016/j.iswa.2025.200524.
3. K. Rehman and G. Jain, "Detection of copy-move forgery with deep CNN features using machine learning classifier," Procedia Computer Science, vol. 258, pp. 3062-3071, 2025, doi: 10.1016/j.procs.2025.04.564.
4. P. Deb and N. Kar, "Satyachitra: CNN-based framework for robust copy-move forgery detection and dataset analysis," Procedia Computer Science, vol. 283, pp. 1103-1112, 2026.
5. H.-S. Chen, Y. Zhu, C.-A. Yu, R. Salloum and C.-C. J. Kuo, "GIFT: A green image forgery tracker," Journal of Visual Communication and Image Representation, vol. 118, p. 104797, 2026, doi: 10.1016/j.jvcir.2026.104797.
6. R. Salloum, Y. Ren and C.-C. J. Kuo, "Image splicing localization using a multi-task fully convolutional network (MFCN)," Journal of Visual Communication and Image Representation, vol. 51, pp. 201-209, 2018, doi: 10.1016/j.jvcir.2018.01.010.
7. J. Dong, W. Wang and T. Tan, "CASIA image tampering detection evaluation database," in Proc. IEEE China Summit and International Conference on Signal and Information Processing (ChinaSIP), 2013.
8. Y.-F. Hsu and S.-F. Chang, "Detecting image splicing using geometry invariants and camera characteristics consistency," in Proc. IEEE International Conference on Multimedia and Expo (ICME), 2006.
9. M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. International Conference on Machine Learning (ICML), 2019.
10. N. Krawetz, "A picture's worth: digital image analysis and forensics," Black Hat Briefings, 2007.
11. R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in Proc. IEEE International Conference on Computer Vision (ICCV), 2017.
12. ductai199x, "image-manipulation-dataset-compilation," Hugging Face dataset, https://huggingface.co/datasets/ductai199x/image-manipulation-dataset-compilation.
13. D. J. Bernstein, N. Duif, T. Lange, P. Schwabe and B.-Y. Yang, "High-speed high-security signatures," Journal of Cryptographic Engineering, vol. 2, no. 2, pp. 77-89, 2012.
14. Coalition for Content Provenance and Authenticity, "C2PA technical specification," https://c2pa.org.

15. A. C. Popescu and H. Farid, "Exposing digital forgeries by detecting traces of resampling," IEEE Transactions on Signal Processing, vol. 53, no. 2, pp. 758-767, 2005.
16. A. C. Popescu and H. Farid, "Exposing digital forgeries in color filter array interpolated images," IEEE Transactions on Signal Processing, vol. 53, no. 10, pp. 3948-3959, 2005.
17. J. Lukáš, J. Fridrich and M. Goljan, "Digital camera identification from sensor pattern noise," IEEE Transactions on Information Forensics and Security, vol. 1, no. 2, pp. 205-214, 2006, doi: 10.1109/TIFS.2006.873602.
18. B. Mahdian and S. Saic, "Using noise inconsistencies for blind image forensics," Image and Vision Computing, vol. 27, no. 10, pp. 1497-1503, 2009, doi: 10.1016/j.imavis.2009.02.001.
19. H. Farid, "Exposing digital forgeries from JPEG ghosts," IEEE Transactions on Information Forensics and Security, vol. 4, no. 1, pp. 154-160, 2009, doi: 10.1109/TIFS.2008.2012215.
20. T. Bianchi and A. Piva, "Image forgery localization via block-grained analysis of JPEG artifacts," IEEE Transactions on Information Forensics and Security, vol. 7, no. 3, pp. 1003-1017, 2012, doi: 10.1109/TIFS.2012.2187516.
21. J. Fridrich, D. Soukal and J. Lukáš, "Detection of copy-move forgery in digital images," in Proc. Digital Forensic Research Workshop, Cleveland, OH, USA, 2003.
22. I. Amerini, L. Ballan, R. Caldelli, A. Del Bimbo and G. Serra, "A SIFT-based forensic method for copy-move attack detection and transformation recovery," IEEE Transactions on Information Forensics and Security, vol. 6, no. 3, pp. 1099-1110, 2011, doi: 10.1109/TIFS.2011.2129512.
23. V. Christlein, C. Riess, J. Jordan, C. Riess and E. Angelopoulou, "An evaluation of popular copy-move forgery detection approaches," IEEE Transactions on Information Forensics and Security, vol. 7, no. 6, pp. 1841-1854, 2012, doi: 10.1109/TIFS.2012.2218597.
24. D. Cozzolino, G. Poggi and L. Verdoliva, "Splicebuster: A new blind image splicing detector," in Proc. IEEE International Workshop on Information Forensics and Security (WIFS), 2015, pp. 1-6, doi: 10.1109/WIFS.2015.7368565.
25. J. Fridrich and J. Kodovský, "Rich models for steganalysis of digital images," IEEE Transactions on Information Forensics and Security, vol. 7, no. 3, pp. 868-882, 2012, doi: 10.1109/TIFS.2012.2190402.
26. B. Bayar and M. C. Stamm, "Constrained convolutional neural networks: A new approach towards general purpose image manipulation detection," IEEE Transactions on Information Forensics and Security, vol. 13, no. 11, pp. 2691-2706, 2018.
27. P. Zhou, X. Han, V. I. Morariu and L. S. Davis, "Learning rich features for image manipulation detection," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 1053-1061, doi: 10.1109/CVPR.2018.00116.
28. M. Huh, A. Liu, A. Owens and A. A. Efros, "Fighting fake news: Image splice detection via learned self-consistency," in Proc. European Conference on Computer Vision (ECCV), 2018, pp. 101-117.
29. Y. Wu, W. Abd-Almageed and P. Natarajan, "BusterNet: Detecting copy-move image

forgery with source/target localization," in Proc. European Conference on Computer Vision (ECCV), 2018, pp. 168-184.

30. Y. Wu, W. AbdAlmageed and P. Natarajan, "ManTra-Net: Manipulation tracing network for detection and localization of image forgeries with anomalous features," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2019, pp. 9543-9552, doi: 10.1109/CVPR.2019.00977.
31. X. Bi, Y. Wei, B. Xiao and W. Li, "RRU-Net: The ringed residual U-Net for image splicing forgery detection," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2019, doi: 10.1109/CVPRW.2019.00010.
32. X. Liu, Y. Liu, J. Chen and X. Liu, "PSCC-Net: Progressive spatio-channel correlation network for image manipulation detection and localization," IEEE Transactions on Circuits and Systems for Video Technology, vol. 32, no. 11, pp. 7505-7517, 2022, doi: 10.1109/TCSVT.2022.3189545.
33. X. Chen, C. Dong, J. Ji, J. Cao and X. Li, "Image manipulation detection by multi-view multi-scale supervision," in Proc. IEEE/CVF International Conference on Computer Vision (ICCV), 2021, pp. 14185-14193.
34. M.-J. Kwon, S.-H. Nam, I.-J. Yu, H.-K. Lee and C. Kim, "Learning JPEG compression artifacts for image manipulation detection and localization," International Journal of Computer Vision, vol. 130, 2022, doi: 10.1007/s11263-022-01617-5.
35. J. Wang, Z. Wu, J. Chen, X. Han, A. Shrivastava, S.-N. Lim and Y.-G. Jiang, "ObjectFormer for image manipulation detection and localization," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 2364-2373, doi: 10.1109/CVPR52688.2022.00240.
36. F. Guillaro, D. Cozzolino, A. Sud, N. Dufour and L. Verdoliva, "TruFor: Leveraging all-round clues for trustworthy image forgery detection and localization," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 20606-20615, doi: 10.1109/CVPR52729.2023.01974.
37. L. Verdoliva, "Media forensics and deepfakes: An overview," IEEE Journal of Selected Topics in Signal Processing, vol. 14, no. 5, pp. 910-932, 2020, doi: 10.1109/JSTSP.2020.3002101.
38. M. A. Qureshi and M. Deriche, "A bibliography of pixel-based blind image forgery detection techniques," Signal Processing: Image Communication, vol. 39, pp. 46-74, 2015.
39. M. C. Stamm, M. Wu and K. J. R. Liu, "Information forensics: An overview of the first decade," IEEE Access, vol. 1, pp. 167-200, 2013, doi: 10.1109/ACCESS.2013.2260814.
40. N. B. A. Warif, A. W. A. Wahab, M. Y. I. Idris, R. Ramli, R. Salleh, S. Shamshirband and K.-K. R. Choo, "Copy-move forgery detection: Survey, challenges and future directions," Journal of Network and Computer Applications, vol. 75, pp. 259-278, 2016, doi: 10.1016/j.jnca.2016.09.008.
41. G. Muhammad, M. H. Al-Hammadi, M. Hussain and G. Bebis, "Image forgery detection using steerable pyramid transform and local binary pattern," Machine Vision and Applications, vol. 25, no. 4, pp. 985-995, 2014, doi: 10.1007/s00138-013-0547-4.
42. T.-T. Ng and S.-F. Chang, "A data set of authentic and spliced image blocks," Columbia

University, ADVENT Technical Report 203-2004-3, June 2004.

43. D. Tralić, I. Župančić, S. Grgić and M. Grgić, "CoMoFoD - New database for copy-move forgery detection," in Proc. 55th International Symposium ELMAR, Zadar, Croatia, 2013, pp. 49-54.
44. B. Wen, Y. Zhu, R. Subramanian, T.-T. Ng, X. Shen and S. Winkler, "COVERAGE - A novel database for copy-move forgery detection," in Proc. IEEE International Conference on Image Processing (ICIP), 2016, pp. 161-165.
45. T. Gloe and R. Böhme, "The 'Dresden Image Database' for benchmarking digital image forensics," in Proc. ACM Symposium on Applied Computing (SAC), 2010, pp. 1584-1590.
46. K. He, X. Zhang, S. Ren and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770-778.
47. A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in Proc. International Conference on Learning Representations (ICLR), 2021.
48. O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in Proc. Medical Image Computing and Computer-Assisted Intervention (MICCAI), 2015, pp. 234-241.
49. I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. International Conference on Learning Representations (ICLR), 2019.
50. H. R. Hasan and K. Salah, "Combating deepfake videos using blockchain and smart contracts," IEEE Access, vol. 7, pp. 41596-41606, 2019.

APPENDIX A – SAMPLE CODE

A.1 Trained detector inference (backend/core/trained_detector.py)

```
def predict(file_path: str) -> dict | None:
    """Return {"p_fake": float, "crops": int, "model": str} or None when no detector
is installed."""
    if not available():
        return None
    import torch
    from PIL import Image, ImageFile
    ImageFile.LOAD_TRUNCATED_IMAGES = True
    img = Image.open(file_path)
    arr = _input_map(img)
    ch = _state["meta"]["channels"]
    mean = np.tile(np.array([0.485, 0.456, 0.406], np.float32), 2)[:ch]
    std = np.tile(np.array([0.229, 0.224, 0.225], np.float32), 2)[:ch]
    crops = _crops(arr)
    x = (np.stack([np.ascontiguousarray(c) for c in crops]).astype(np.float32) /
255.0 - mean) / std
    t = torch.from_numpy(x).permute(0, 3, 1, 2).to(_state["device"])
    with torch.no_grad():
        lg = _state["model"](t).squeeze(1).float()
    return {"p_fake": float(torch.sigmoid(lg.mean())), "crops": len(crops), "model":
_state["meta"].get("name", _state["meta"]["file"])}
```

A.2 Weighted fusion and verdict bands (backend/core/decision_engine.py)

```
AGENT_WEIGHTS = {
    "metadata": 0.20,
    "image_forensics": 0.30,
    "security": 0.25,     # C2PA + watermark provenance (Phase 5)
}

# Verdict thresholds on the authenticity score.
AUTHENTIC_THRESHOLD = 0.75
FAKE_THRESHOLD = 0.40
def aggregate_score(results):
    """Weighted authenticity average across agents.

    Agents may report confidence=None (e.g. security agent when no provenance
    data exists either way) — those are excluded rather than counted as 0.5.
    """
    total_w = 0.0
    acc = 0.0
    for r in results:
        conf = r.get("confidence", 0.5)
        if conf is None:
            continue
        w = AGENT_WEIGHTS.get(r.get("agent"), 0.25)
        acc += float(conf) * w
        total_w += w
    return acc / total_w if total_w > 0 else 0.5



def verdict_from_score(score):
    if score >= AUTHENTIC_THRESHOLD:
        return "Authentic"
    if score <= FAKE_THRESHOLD:
        return "Fake"
    return "Inconclusive"
```

A.3 Training step of the detector (training/train_image_detector.py)

```
def crops_of(arr, k, rng=None, train=False):
    """Crops of size CROP. Train: k random crops. Test: a deterministic grid (at most
k)."""
    h, w, _ = arr.shape
    if h < CROP or w < CROP:
        arr = np.pad(arr, ((0, max(0, CROP - h)), (0, max(0, CROP - w)), (0, 0)),
mode="reflect")
        h, w, _ = arr.shape
```

```
    if train:
        res = []
        for _ in range(k):
            y, x = rng.randint(0, h - CROP), rng.randint(0, w - CROP)
            c = arr[y:y + CROP, x:x + CROP]
            if rng.random() < .5:
                c = c[:, ::-1]
            if rng.random() < .5:
                c = c[::-1]
            res.append(c)
        return res
    ys = np.linspace(0, h - CROP, min(k, max(1, (h - CROP) // 64 + 1))).astype(int)
if h > CROP else [0]
    xs = np.linspace(0, w - CROP, min(k, max(1, (w - CROP) // 64 + 1))).astype(int)
if w > CROP else [0]
    grid = [arr[y:y + CROP, x:x + CROP] for y in ys for x in xs]
    if len(grid) > k:
        idx = np.linspace(0, len(grid) - 1, k).astype(int)
        grid = [grid[i] for i in idx]
    return grid
def train_model(arrs, ys, dev, epochs, repeat, bs, lr, seed, val=None, log=""):
    rng = random.Random(seed)
    torch.manual_seed(seed)
    model = make_model(dev)
    opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=1e-4)
    steps = epochs * ((len(arrs) * repeat + bs - 1) // bs)
    sched = torch.optim.lr_scheduler.OneCycleLR(opt, max_lr=lr, total_steps=steps,
pct_start=0.15)
    scaler = torch.amp.GradScaler("cuda")
    lossf = nn.BCEWithLogitsLoss()
    order = list(range(len(arrs))) * repeat
    for ep in range(epochs):
        model.train()
        rng.shuffle(order)
        tot, n = 0.0, 0
        for b in range(0, len(order), bs):
            idx = order[b:b + bs]
            crops, lab = [], []
            for i in idx:
                crops += crops_of(arrs[i], 1, rng, True)
                lab.append(ys[i])
            x = to_tensor(crops, dev)
            y = torch.tensor(lab, dtype=torch.float32, device=dev)
            with torch.autocast("cuda", dtype=torch.float16):
                loss = lossf(model(x).squeeze(1), y)
            opt.zero_grad(set_to_none=True)
            scaler.scale(loss).backward()
            scaler.step(opt)
            scaler.update()
            sched.step()
            tot += loss.item() * len(idx)
            n += len(idx)
        msg = f"{log} epoch {ep + 1}/{epochs} loss {tot / n:.4f}"
        if val is not None:
            pv = predict(model, val[0], dev)
            msg += f"  val acc {((pv >= .5) == np.array(val[1])).mean():.4f}"
        print(msg, flush=True)
    return model
```